

HEARTH

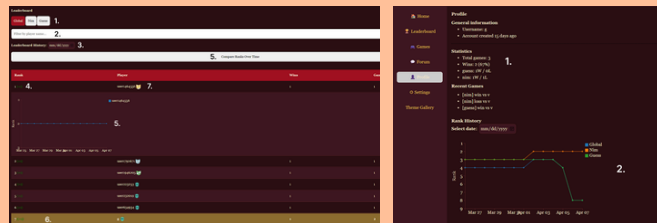
GROUP 401: HENRY, NICK, GEORGE

NEW FEATURES: LEADERBOARD, AUDIO/ANIMATION, THEMES

LEVEL UP YOUR GAMES

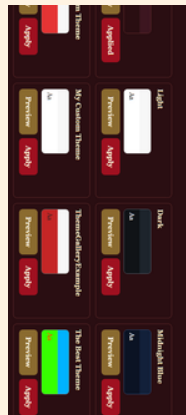
LEADERBOARD:

Curious how you rank against other players? Or how you've been progressing? Or if you're the best in a specific game?



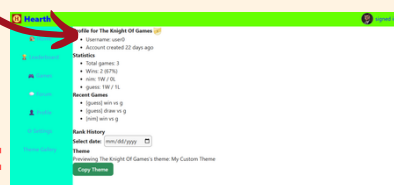
THEMES:

Set your background to match a fiery ember, or the gray shade of charcoal. Maybe even steal your friends design?



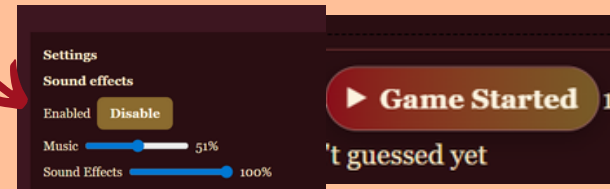
W H Y

To improve engagement in multiplayer games!



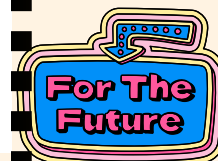
AUDIO + ANIMATIONS:

Wouldn't you enjoy some cool customizable music site-wide? And even game-specific sound effects and animations?



HOW?:

- React frontend and Node.js/Express backend with MongoDB
- Leaderboard is computed dynamically from stored data
- Theme system is applied through global CSS variables
- Audio/animation system is managed through centralized contexts using libraries (Howler.js, Motion/CSS)



- Seasonal ranks, themes, and audio profiles!
- Game additions
- A friends system



GITHUB.COM/NEU-CS4530/SPRING26-PROJECT-SPRING26-PROJECT-GROUP-401



SPRING26-PROJECT-SPRING26-PROJECT-GROUP-GGHK.ONRENDER.COM

ABOUT THE PROJECT

EloSmash transforms GameNite from a casual multiplayer platform into a competitive gaming ecosystem. Players compete in ranked matches of **Nim** and **Number Guesser**, with an adaptive Elo rating system, skill-based matchmaking, percentile-based rank tiers, per-game leaderboards, and rich user profiles with competitive statistics and rating history.

KEY FEATURES

- ✕

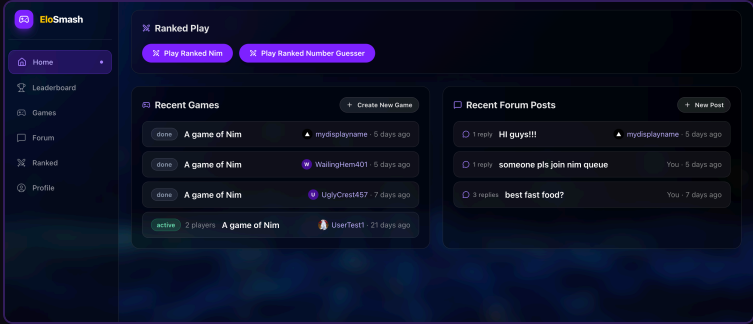
Adaptive Elo Rating System
Per-game ratings with provisional K-factor (48→32) for faster new-player calibration. Server-authoritative updates.
- ⚙️

Skill-Based Matchmaking
Expanding Elo search window (±100 → ±600) balances match quality with queue times.
- 🏆

Rank Tiers & Leaderboards
Bronze **Silver** **Gold** **Diamond** **GM** — percentile-based with podium for top 3.
- 👤

Enhanced User Profiles
Elo history graphs, match history, win/loss records, streaks, profile pictures, and editable display names.
- 🔒

Match Integrity & Auth0
30s disconnect and 120s inactivity forfeits, navigation blocking, ranked rematches, and JWT-based authentication.



Home Page — Ranked play buttons, recent games, and forum posts



Profile Page — Rank badges, Elo graph, win/loss stats, and match history

TEAM CONTRIBUTIONS

Neil Agrawal

Database migration, matchmaking, rank tiers, Auth0 integration, deployment, UI polish

Matthew Tung

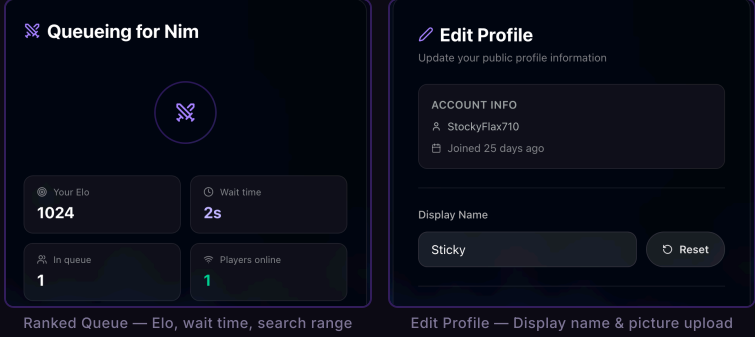
Ranked queue UI, matchmaking range, queue display, profile pictures, rematches, deployment pipeline

Justin Gubbens

Elo algorithm, rating model, post-match updates, profile expansion, match history, data visualization

Ernest Chae

File hosting, anti-cheat checks, leaderboard system, forfeit enforcement, persistence testing pipeline



Ranked Queue — Elo, wait time, search range

Edit Profile — Display name & picture upload



Leaderboard — Per-game rankings with gold, silver, and bronze podium

TECHNOLOGY STACK

FRONTEND

React Vite Tailwind CSS

Socket.IO Axios Recharts

Framer Motion Auth0 SDK

BACKEND

Express Socket.IO MongoDB

Zod Auth0 JWT TypeScript

Node.js

TESTING & CI

Vitest Playwright Supertest GitHub Actions

DESIGN DECISIONS

- TypeScript monorepo** with a shared contract package ensures compile-time type safety across client, server, and socket events.
- Decoupled Podium & Rankings** Top 3 players fetch independently so the leaderboard podium loads instantly even if the full list is slow.
- Matchmaking and queue logic** are separated into independent services for testability. An atomic claim operation prevents race conditions during concurrent match attempts.
- Three Parallel GET Requests** Profile loads user info, rank, and match history independently so the page renders incrementally.

FUTURE WORK

- Elo decay:** Reduce ratings for inactive players to keep leaderboards fresh and incentivize continued play.
- Seasonal resets:** Time-bounded competitive seasons with leaderboard resets and end-of-season rewards.
- Win-rate gating:** Require minimum win rates within a rank tier before promotion, adding depth to the progression system.
- Extended profiles:** User bios, banner images, and additional games with independent Elo tracks and leaderboards.

PrismPlay

Group 403

[Try it here](#)
[Source code](#)

Stephanie Chen, Kaitlyn Hu, Katya Luchette, June Leigh

What are we?

PrismPlay is an interactive gaming website for players of all skill levels. Build your community through friends, private chats, group chats, and scheduled games. Need extra help? Use in-game hints for Checkers and Connect Four to grow your skills and track your progress. You can also play anytime against a computer opponent — with adjustable difficulty levels to match your skills.

How it's made

Connect Four and Checkers were built following existing patterns to maintain clean architecture. Hints and game updates use WebSockets for real-time state sync, with a reusable React component and custom hooks to reduce duplication. Hint and computer opponent algorithms use a priority-based approach optimized for speed. The computer opponent uses a user stub to reuse existing game logic without storing extra data.

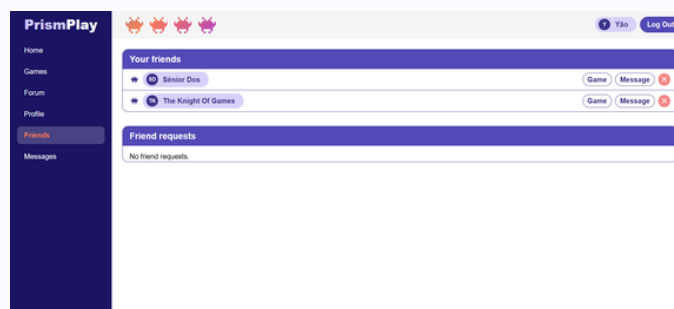
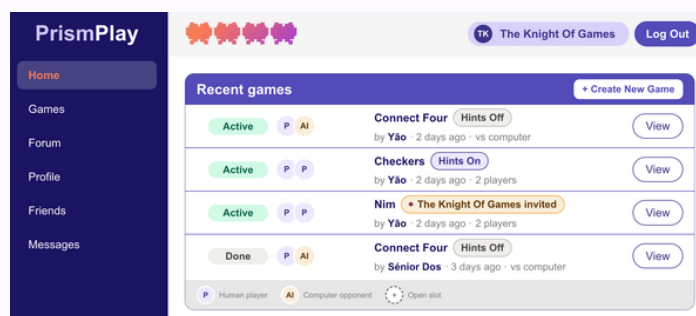
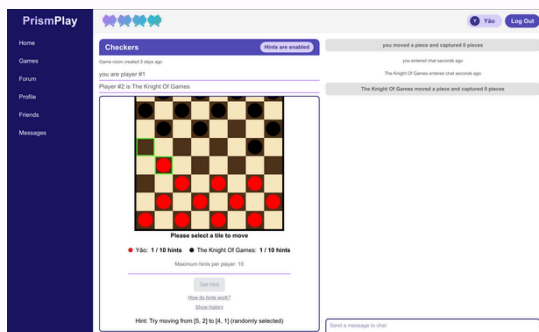
The friends feature uses a REST API with a new repository for connections and requests. Chat was extended to support direct and group messaging via WebSockets, with Cloudinary for image uploads and Linkify for hyperlink parsing. Scheduled games extend existing game records, filtered on the frontend until start time. The UI was modernized with Tailwind CSS.

Tech Stack

- Frontend: React, Tailwind CSS, TypeScript
- Backend: REST APIs & WebSockets
- Database: MongoDB
- Integrations: Cloudinary, Linkify
- Deployment: Render

What's next?

- AI-powered hints — deeper game state explanations to accelerate skill development
- Profile customization — profile pictures, bios, interests, and game history to strengthen community connections
- Leaderboards & player tiers — track top players by win rate and match competitors of similar skill levels to drive engagement



PlayHub!

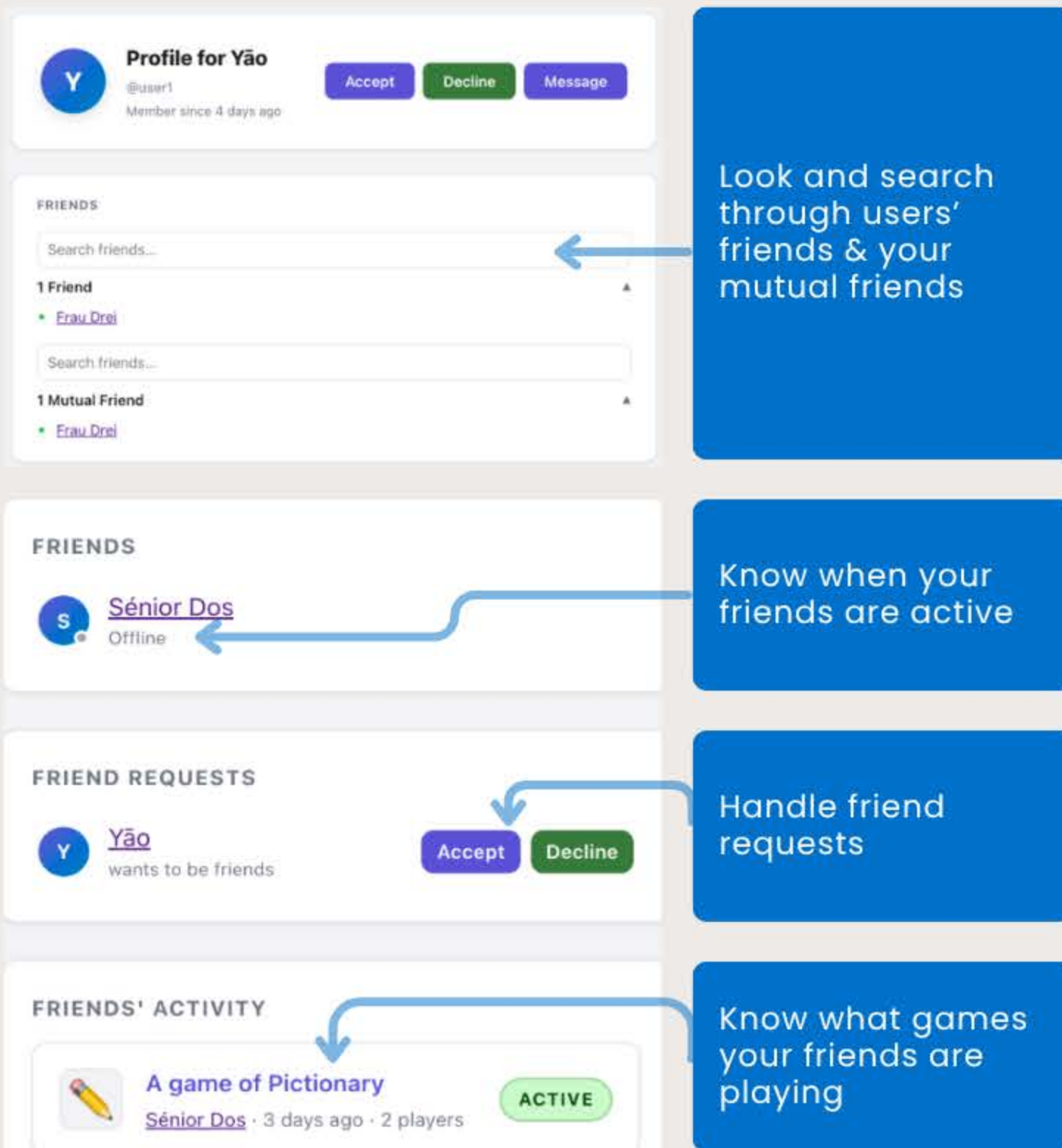
Group 404
Flora Au, Paulina Acosta, Kaushal Jha, Rohin Patel

PlayHub! Is an online platform that aims to improve online connections between users by implementing social features that can create more community: a friend system, a 3+ multiplayer game of Pictionary, and a chatting system.

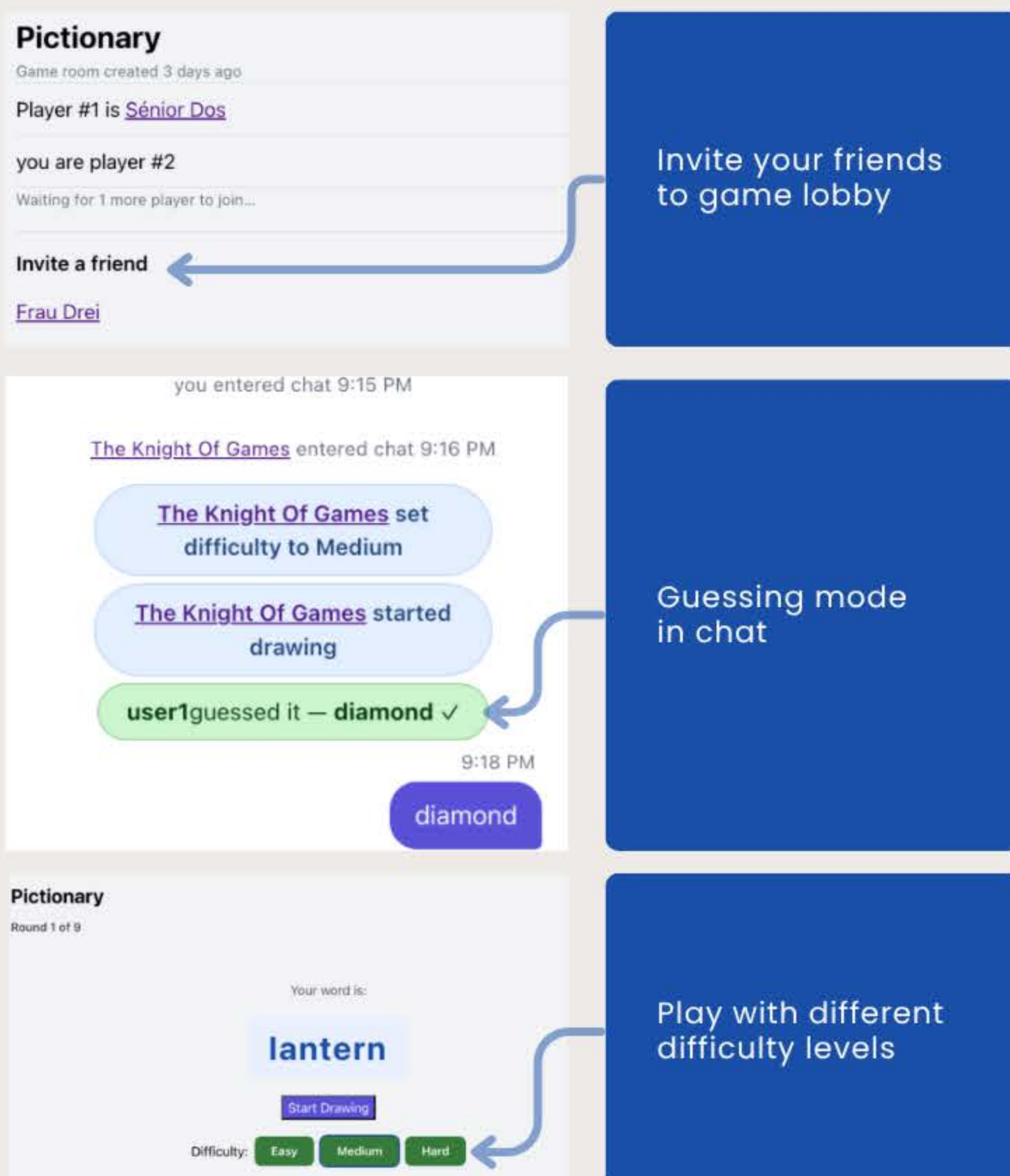
Website: <https://spring26-project-group-404.onrender.com>

Repo: <https://github.com/neu-cs4530/spring26-project-group-404>

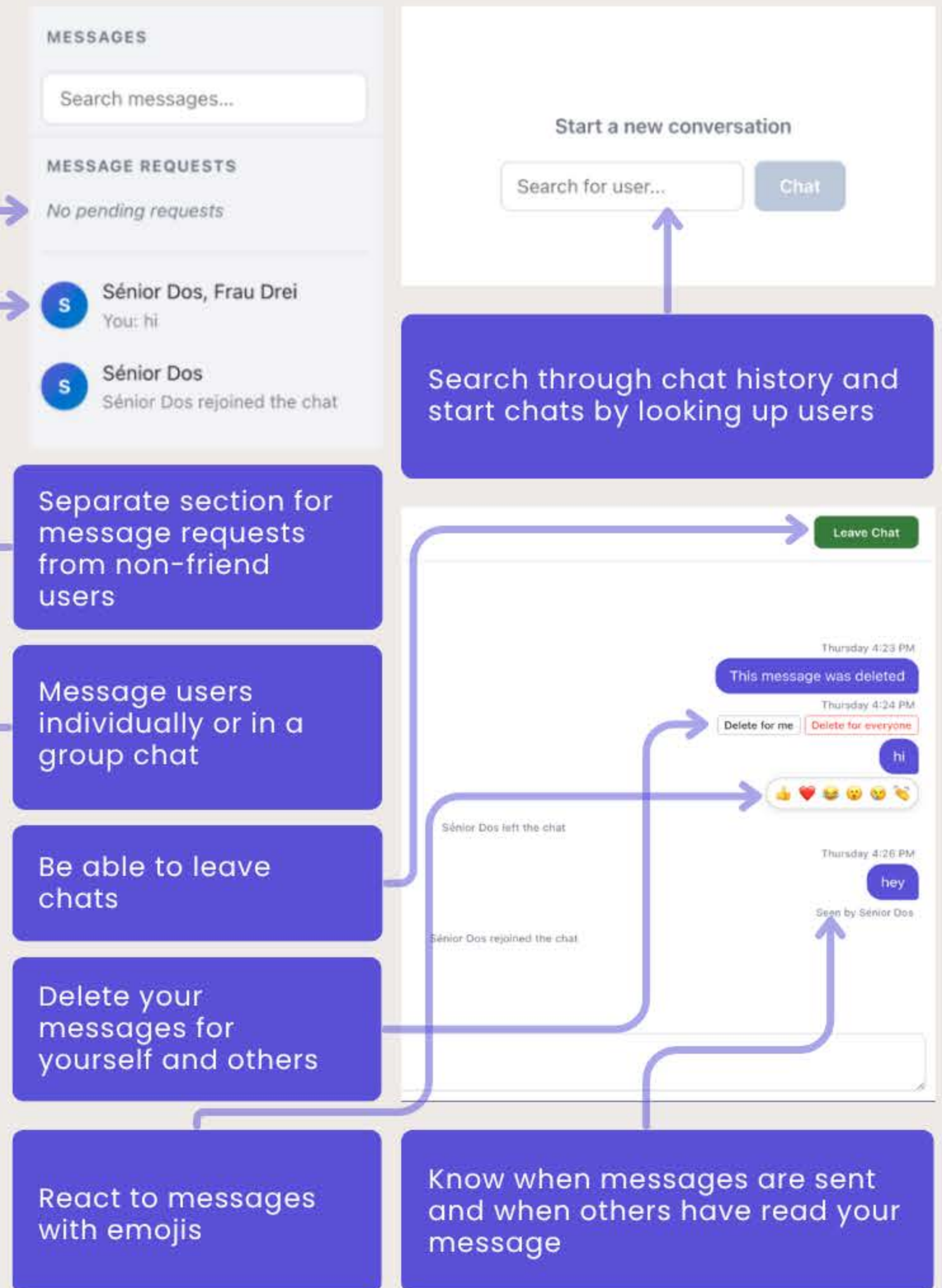
Friend System



Pictionary



Chat System



Technology Stack & Design

- Frontend: React, TypeScript, Socket.io, CSS
- Backend: MongoDB, TypeScript, Node.js, Express, Socket.io
- Testing: Vitest, Playwright

We implemented real-time social and multiplayer features into our PlayHub codebase, utilizing a Node.js backend and WebSockets to sync direct messages, live online statuses, and game states across all connected clients. Friends, conversations, and games are stored in key-value repositories on the server. In addition, Pictionary uses server-side stroke buffering and real-time drawing sync. The platform site is deployed on Render with MongoDB as the database.

What's Next?

For next steps, it would make it more accesible for on the go gaming if we were to make a **mobile app** version of PlayHub. We can also add **more multiplayer games** and have **leaderboards** for friends to expand upon the social aspect of our site.

CS 4530 Final Project: OnlyGames

Team 405: Christian Wu, Albert Lin, David Lee, Eric Nie

What is OnlyGames? OnlyGames is an online game hub that extends of GameNite by adding Replay functionality to review previous games, a new game called Go, and Leaderboards. It allows users to meet up online and play games together.

Game GO:

Get ready for a new game coming to OnlyGames! Featuring Go, straight from Ancient China! Choose between three board sizes depending on game length, and start placing tiles and dominating the battlefield. The game handles enforcing rules, capturing pieces, and calculating the score, so there's no fighting or cheating, just strategy.

Replay:

Missed what just happened? Use the Replay panel to scroll back through earlier moves, step forward/back, and see the game exactly as it was at any moment. Want a do-over? Pick a move and hit Fork to start a new game from that point and try a different strategy.

Leaderboard:

Getting a little competitive? Check out our leaderboard. Rankings are based on a custom ELO formula which weighs wins against higher ranked players more heavily. For each game we track weekly, monthly, yearly, and all-time rankings. When in game, a player's ranking will also be shown alongside their name. For less competitive players or players taking a break, we provide an unranked mode.

Technology Stack

The backend is powered by typescript via Node.js, with database storage using mongoDB. The server is hosted on onRender.

The frontend is powered by React and typescript. Socket.io is used for updating the client

The codebase is built with Vite, with testing via Vitest and Playwright. Formatting was handled by ESLint and Prettier.

Links & Demos

Live Demo:

<https://cs4530-405.onrender.com/>

Source:

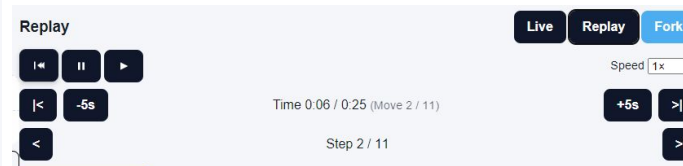
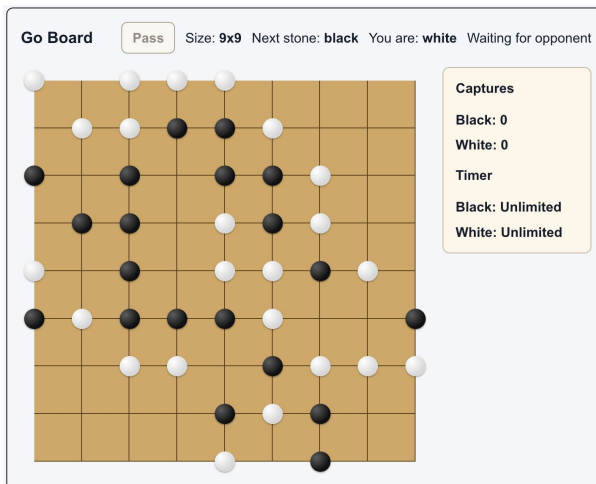
<https://github.com/neu-cs4530/spring26-project-group-405>

Design Decisions

The replay system reconstructs game state by reapplying the first k moves on the server and returning the appropriate view, with previous games stored in MongoDB. The leaderboard stores timestamps for everything but offloads all calculations to the client side, also using MongoDB for storage. Go follows the same client-controller-server model as the other games but with more complex board state and a custom worker for tracking game time out behavior.

Future Work

One day, we would love to add a more robust messaging service to our games platform. This would be similar to sub-Reddits for each game, allowing like minded users to discuss strategy and learn from each other. The goal of a messaging service would be to allow users to form long term, meaningful bonds with each other, while also maintaining a level of moderation.



 OnlyGames

Leaderboard

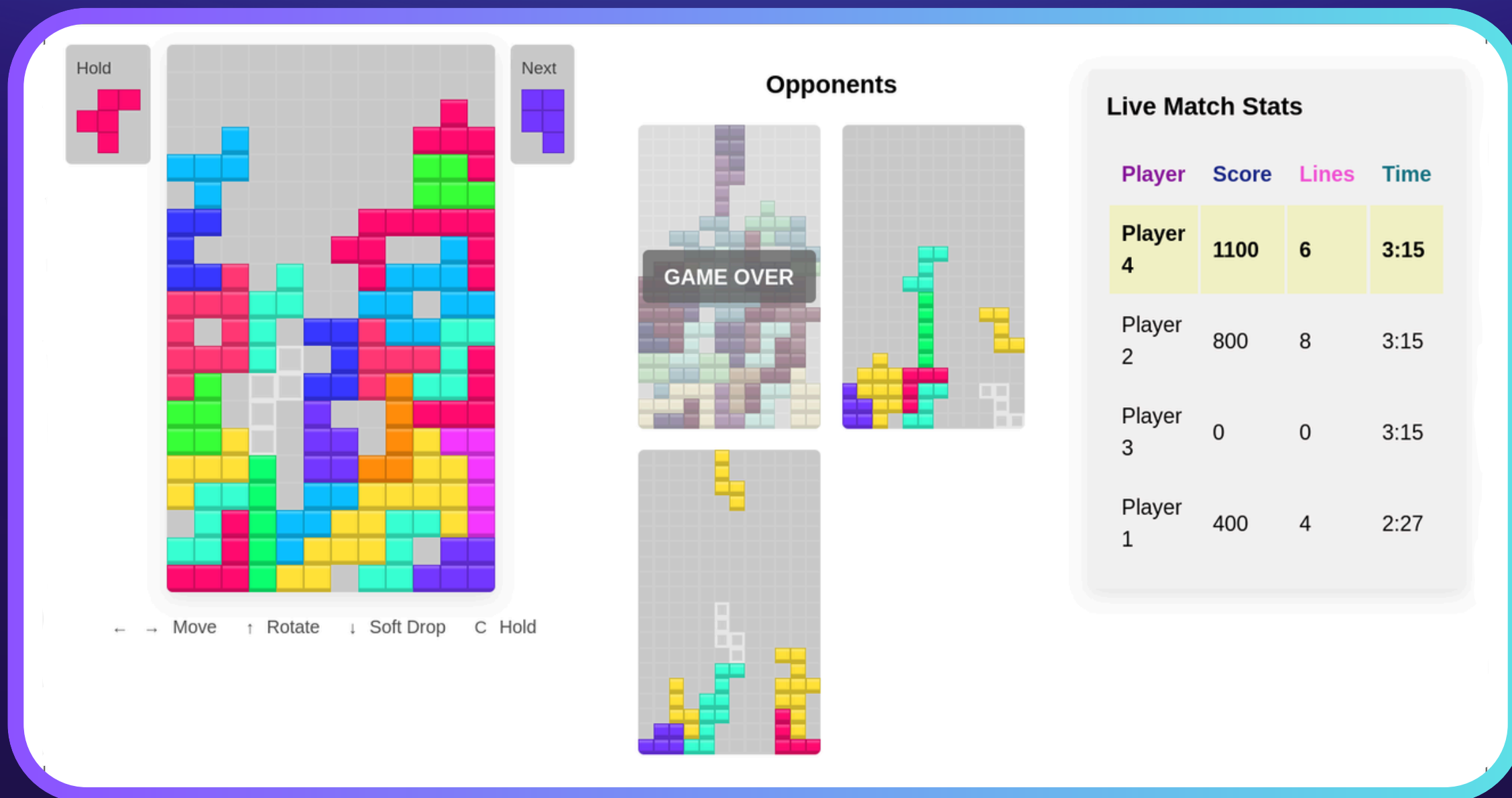
Rankings are sorted by ELO and can be viewed by weekly, monthly, yearly, or all-time splits. Rank movement reflects the last 24 hours within the selected period.

GuessGame			NimGame			GoGame		
Weekly			Monthly			Yearly		
All-Time								
RANK	PLAYER	ELO	RANK CHANGE (24H)	WINS	LOSSES	TIES	WIN %	
1	gooner123	111	0	3	2	0	60.0%	
2	Yão	106	0	3	3	0	50.0%	
3	0	100	0	0	0	0	0.0%	
4	1	100	0	0	0	0	0.0%	

PETRIS

COMPETITIVE MULTIPLAYER PENTOMINO BATTLE

Compete against up to 3 other players in real time in this Tetris-like block game. Clear lines to earn points, and survive as long as you can to claim ultimate victory!



FEATURES

- Real-time multiplayer
- Pentomino pieces
- 3 difficulty levels
- Achievements
- Leaderboards
- Score tracking
- Ghost piece
- Next piece preview

TECH STACK

Game Platform & Websockets

Built on the GameNite platform, Petris uses websockets to transmit move information live.

Real-Time Gameplay

The game engine on the backend accepts all the typical moves - left, right, rotate - but also accepts a special "tick" move to drop pieces down the board. The server automatically transmits ticks on a time interval, with the speed increasing as the game progresses.

Player Progress & Data Management

In addition to the game itself, we added a leaderboard system and achievement system. Both of these store data in our MongoDB database, to track player progress between sessions.

WHAT'S NEXT?

- Extensions to the Petris project could include new features such as an Elo rating system or the ability to play with a friend via split-screen multiplayer.
- Additionally, new real-time competitive games could be added to GameNite leveraging the web sockets design, such as a slither.io type game.

Play Live: <https://spring26-project-spring26-project-group-h54z.onrender.com/>

See the Code: <https://github.com/neu-cs4530/spring26-project-spring26-project-group-406-1>

Group 406 - Zelen Gungor - Nicholas Rising - Vainavi Vuppu - William Turner

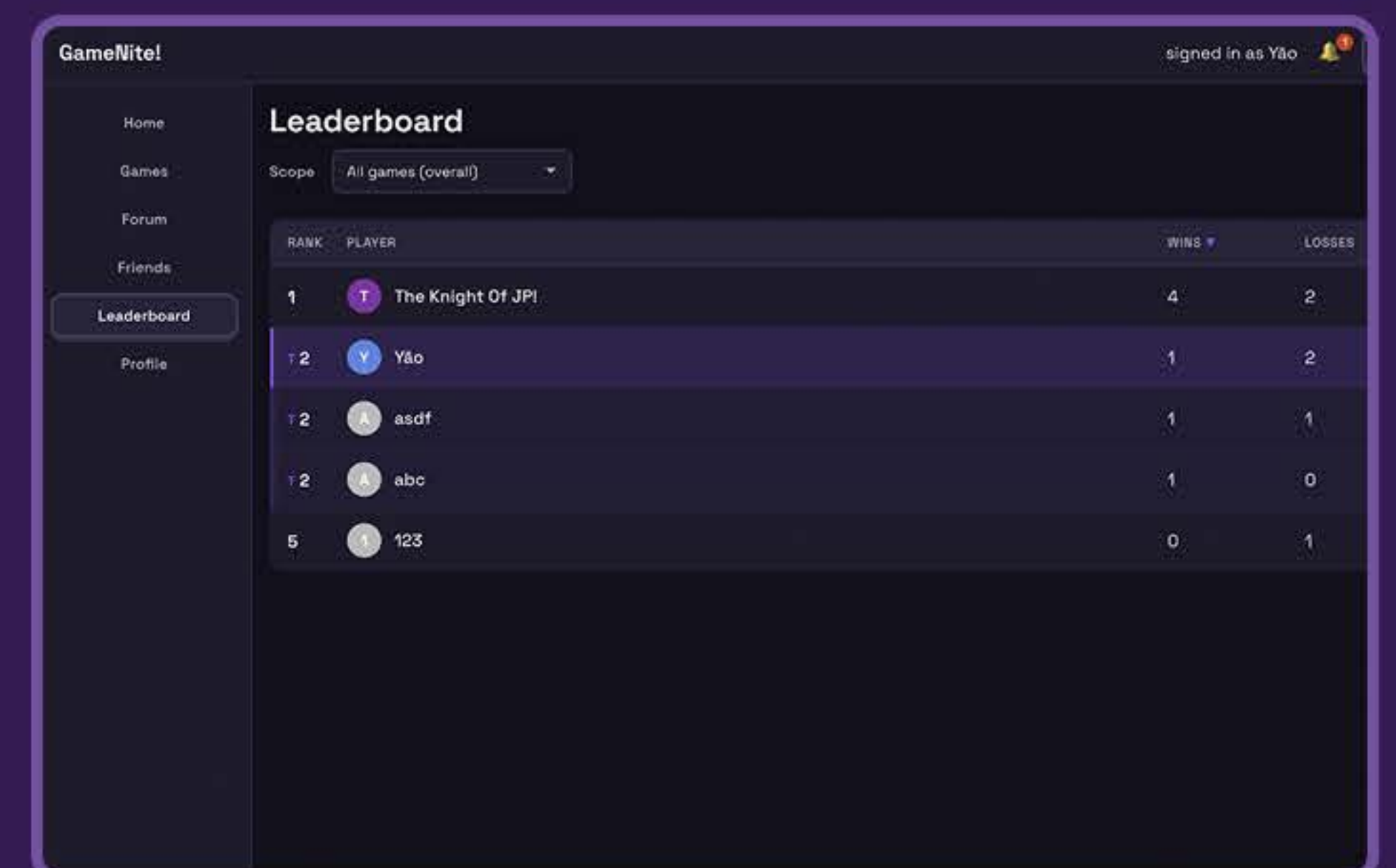
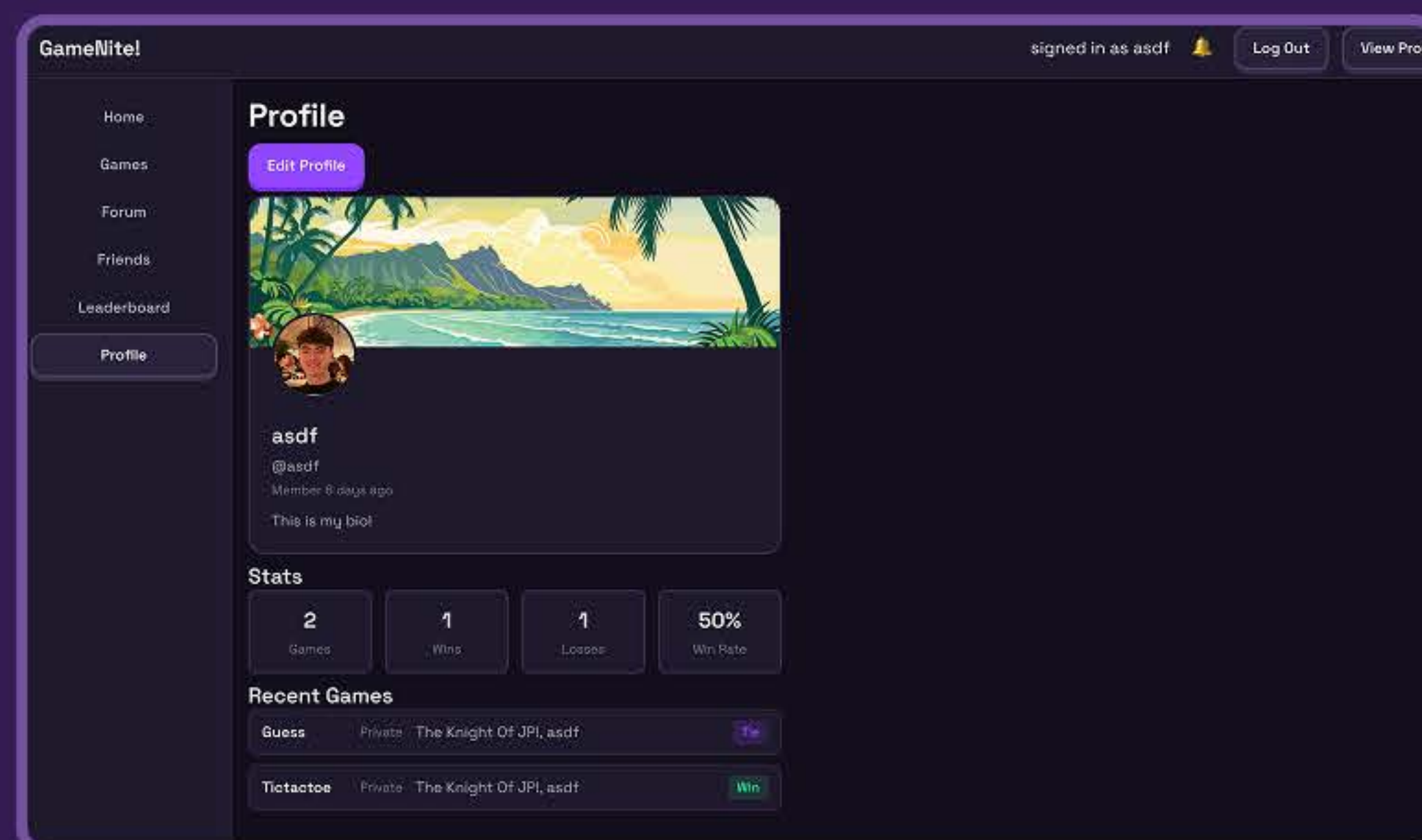
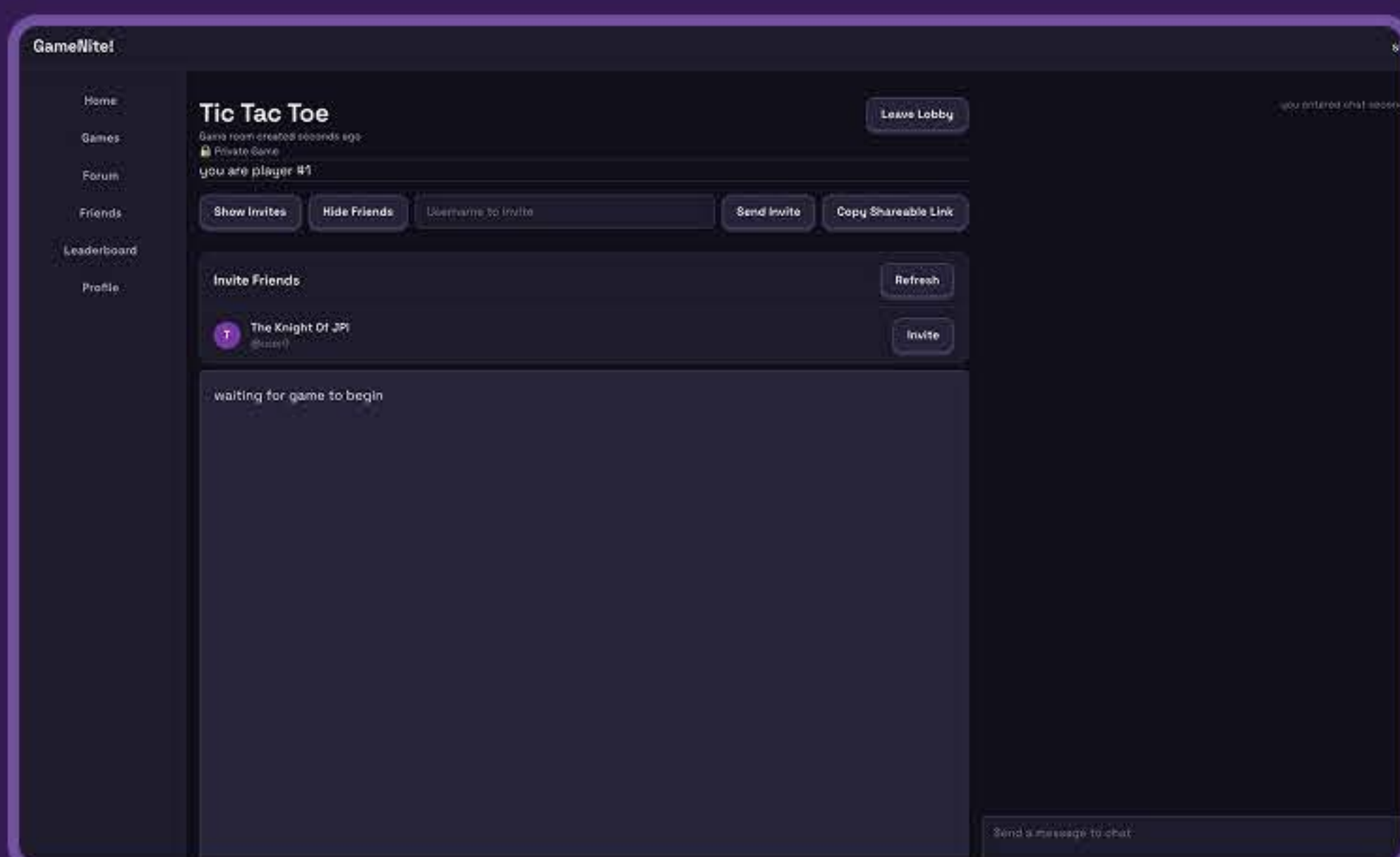


CS4530 Final Project: GameNite

Group 407: JP Norton, Jack Oehling, Garrick Cheng, Liam Carvajal

Features & Description

GameNite is a multiplayer web app that combines social features with gameplay. Users can customize profiles with avatars, banners, bios, and privacy settings, build friendships through friend requests and live online presence, create invite only private game lobbies with shareable join links and real time notifications, and compete on persistent leaderboards based on valid completed matches. We also added Tic-Tac-Toe and MineFinder to expand the game library.



Tech Stack

Built with React and TypeScript on the frontend and Express plus Socket.io on the backend. Data is persisted with MongoDB and Keyv, client and server stay aligned through shared TypeScript types, GitHub Actions handles CI, Playwright covers end to end testing, and the deployed site is hosted on Render.

Design Decisions

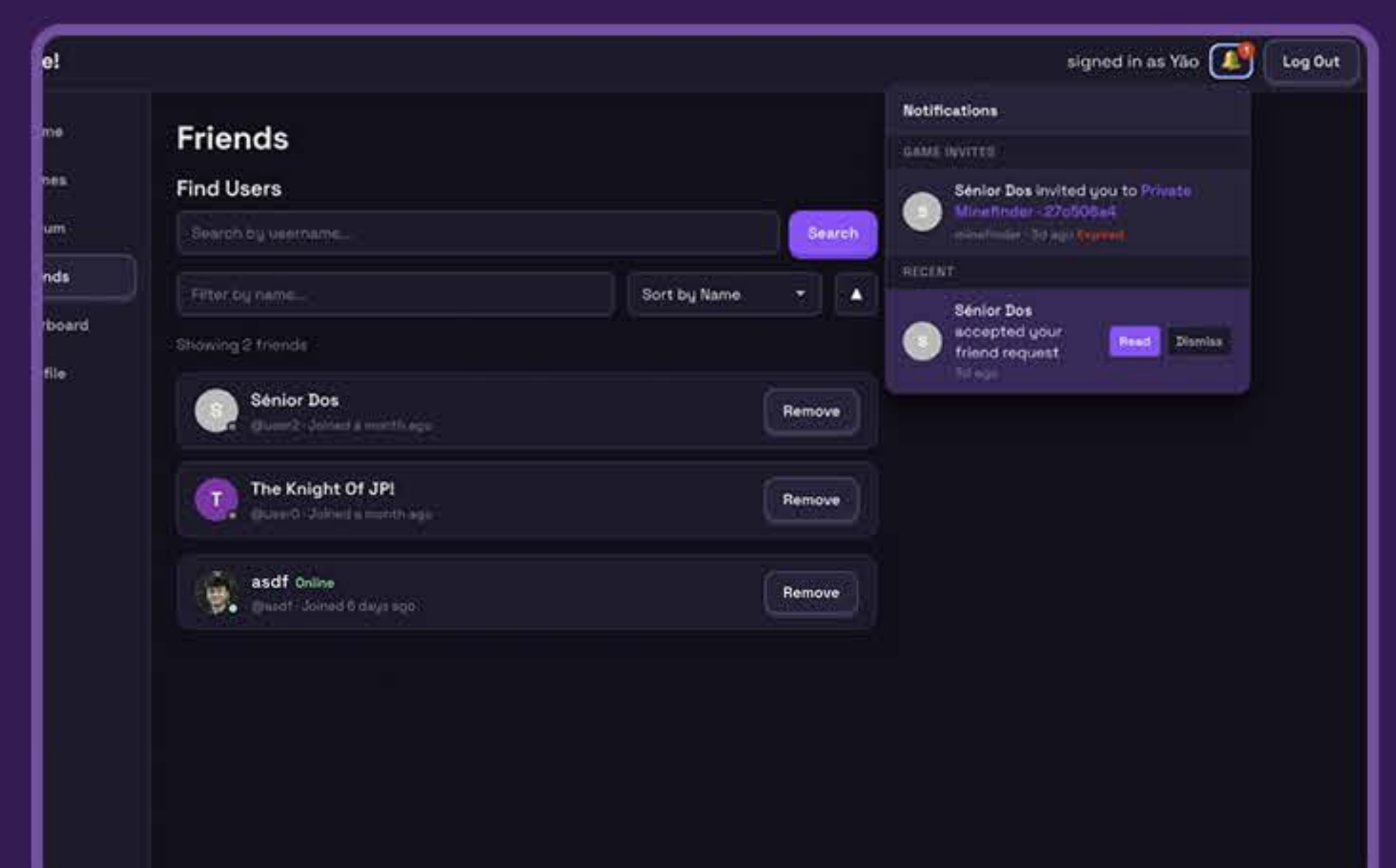
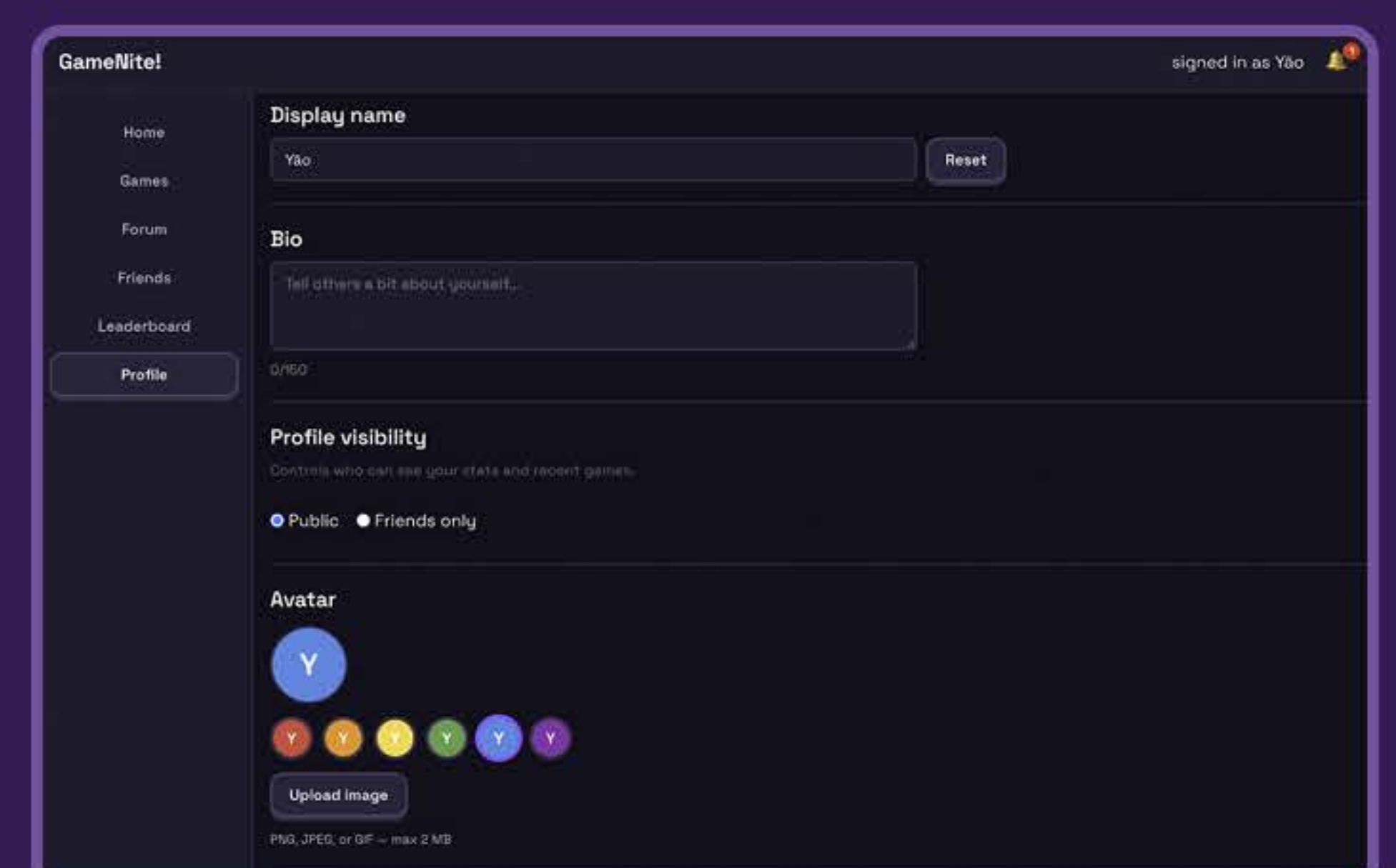
We separated backend logic into repository, service, and controller layers to make the system easier to test and maintain. On the client, we used a hooks plus services pattern to keep components cleaner and reduce clutter from state and API logic. We also used a single shared socket connection for real time presence, invite updates, and notifications, and enforced private game access on the server for stronger security.

Repo

<https://github.com/neu-cs4530/spring26-project-team-407>

View Project Demo Site

<https://spring26-project-team-407.onrender.com>



Future Work

- direct messaging between friends
- suggested friends based on mutual connections
- password protected private games
- more games to play
- continued UI and performance improvements

CS4530 Final Project Poster

What we added

We felt that the existing GameNite platform did not have the appropriate features to keep users engaged and keep them coming back to the site. The site lacked a shared resource that users could connect with each other through.

We implemented 3 user stories that helped solve this problem and connect the GameNite users together. First, we implemented a **ranking system** that reflects a user's skill level for a specific game type along with an overall rank which are all displayed on a searchable leaderboard.

Next, we implemented a **friend system** that allows users to connect with friends, and make new ones in the GameNite community. The friend system allows users to easily keep track of their friends' stats/rankings along with the functionality to play private games with their friends.

Lastly, we implemented public **user statistics** so that users could track their own performance along with other users on the platform. These are displayed on users' profiles.

Demo & Source Code

Live Demo Site: <https://spring26-project-team-408-1.onrender.com/>

Source Code: <https://github.com/neu-cs4530/spring26-project-team-408-1>

By Team 408: Jack Mitchell, John Young, Peter Moise, and Maxwell Hwee

Stats

Overall record 6W – 6L – 0T 12 played	Overall win rate 50% across all games	Overall rank 1277 Across all games	Last 5 games 3W – 2L – 0T 60% recent win rate
--	--	---	--



GAME	RECORD	WIN %	RANK
nim	3W – 3L – 0T	50%	1277 best
guess	3W – 3L – 0T	50%	1277 worst

Leaderboard

	Nim	Guess	Overall
Your Position: 15 (Games: 12)			
#	Player		Rank
1	Kate		1350
2	Liam		1344
3	Mia		1338

Incoming Requests

Bob (Friends with user0, pending to user0 (@user1))

Accept Decline Block

Outgoing Requests

No outgoing requests.

Friends

Frank (Friends with user0, pending from user7 (@user5))

Remove Block

Grace (Friends with user0 (@user6))

Remove Block

Blocked Users

No blocked users.

Future Additions

In the future, it would be nice to implement a more robust user search feature to allow for users to more easily find people to connect with. As well, building out the ranking system to be more advanced in the calculation - also displaying different tiers of ranks to give users a progression goal. Lastly, to expand the user statistics we could give individual game statistics, allowing users to look into user's most recent games.

Technical Design Choices

Ranks/Leaderboard:

- Every completed game triggers logic that calculates a new rank for each user involved in that game. The rank is based on opponent rankings, games played, and existing rank. Each users rank data is stored in a new rank repo in our DB.

Friend System:

- Friendship relationships are stored in a new repo and are maintained by our backend friendship service. There are new API endpoints and the front end service displays all friend information. When friends, users can start private games, hidden from public view.

User Stats:

- Game results are stored in a new repo in the MongoDB database with a record for each user in each game. This allows us to have specific access to each game result for future game statistics additions. The results are pulled in a react hook and sorted/filtered by user/game-types.

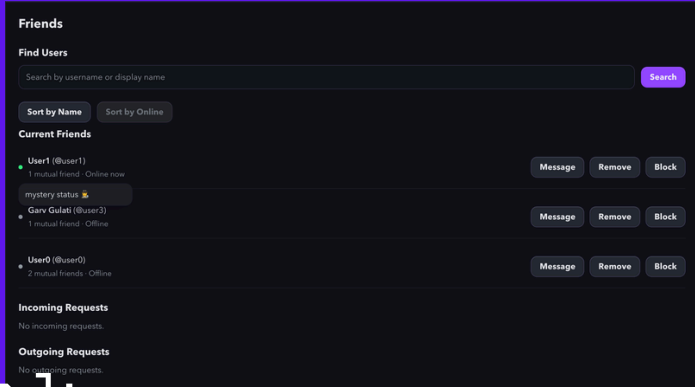
GameNite: Building a Social Gaming Platform

Team 409: Jiaqi, Yash, Garv, Julie

GameNite is a web-based multiplayer gaming platform. However, it lacked essential social functionality, limiting interactions to individual game sessions.

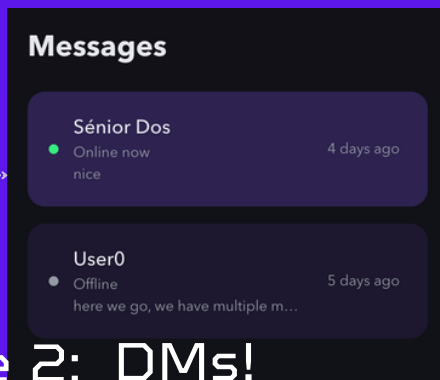
In this project, we extended GameNite by building a friend system, real-time direct messaging, and invite-only game rooms. These additions enable persistent relationships, private communication, and controlled gameplay experiences, turning GameNite into a more complete social gaming platform.

Built with: React + TypeScript, Node.js + Express, MongoDB, WebSockets (real-time updates)



Feature 1: Connect w/ Friends

- Send, accept, and remove friends
- Real-time updates without refresh
- Online/offline indicators

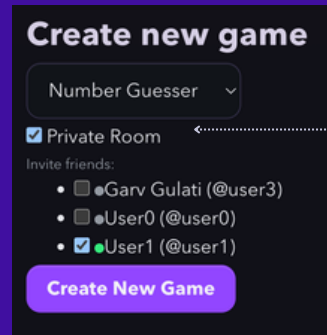


Feature 2: DMs!

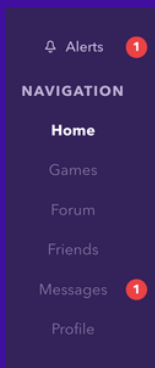
- Private messaging with live updates

Feature 3: Private Rooms

- Create invite-only game rooms
- Control exactly who joins

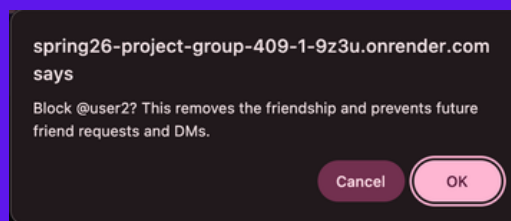


Feature 4: Stay Updated



- Notifications for messages, invites, requests
- Activity feed shows what friends are doing

Feature 5: Feel Safe



- Block or mute users
- Control your interactions

Design

- Real-time architecture using WebSockets
- Secure backend access control (HTTP 403)
- Scalable system design (modular services)

Try it yourself!

[Link to Repo](#)

[Link to website](#)

What's Next?

- Group chats
- Smarter friend recommendations
- Enhanced social features

GameNite Party+

CS 4530 · Spring 2026 · Team 412 · Section 4

Hanchao Ding · Jahan Goel · James Cottrell · Jason Lam

🔗 Live App: spring26-project-group-412-1.onrender.com

🔗 GitHub: [neu-cs4530/spring26-project-group-412-1](https://github.com/neu-cs4530/spring26-project-group-412-1)

WHAT IS GAMENITE PARTY+?

GameNite is a browser-based multiplayer game platform. GameNite Party+ extends it with four new social features: a full **Monopoly board game**, a **real-time friend invite system**, **emoji reactions** in chat and on the board, and **user profile customization**. Together these additions transform GameNite from a quick two-player launcher into a social hub where players can connect, invite friends, and express themselves during gameplay.

FOUR FEATURE AREAS

Monopoly

Full turn-based board game for up to 4 players. Property purchase, rent, houses/hotels, jail, Chance, Community Chest, bankruptcy, and seamless disconnect/rejoin.

Friend Invites

Hosts invite players by username. Recipients see a real-time red badge on their sidebar. Invites show status (pending / accepted / declined / canceled) and auto-expire after 5 minutes.

Emoji Reactions

React to chat messages with emoji bubbles visible to all players in real time. During gameplay, board reactions float above your piece for 3 seconds then fade.

Profile Customization

Upload a photo (PNG/JPEG/WebP, max 2 MB), set a display name and bio. Preview before saving. Changes persist across sessions and appear throughout the app.

KEY DESIGN DECISIONS

Targeted invite delivery

Each socket joins user:{username} on connect, so invites push directly to the recipient with no broadcast overhead.

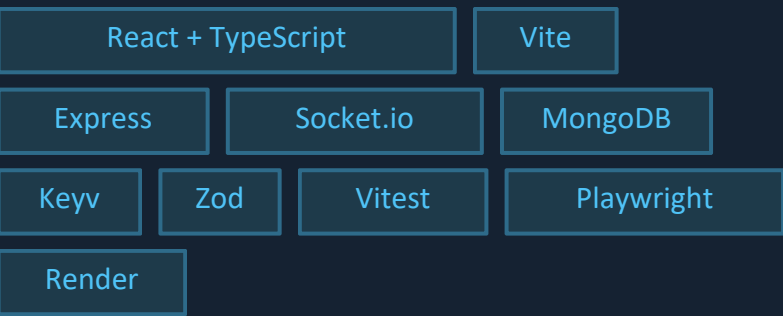
Quick board reactions

Board reactions use a dedicated socket event and are never written to the database, keeping chat history clean.

Shared type system

All Zod validators and TS types live in a shared package imported by both client and server — API contract violations fail at compile time.

TECH STACK

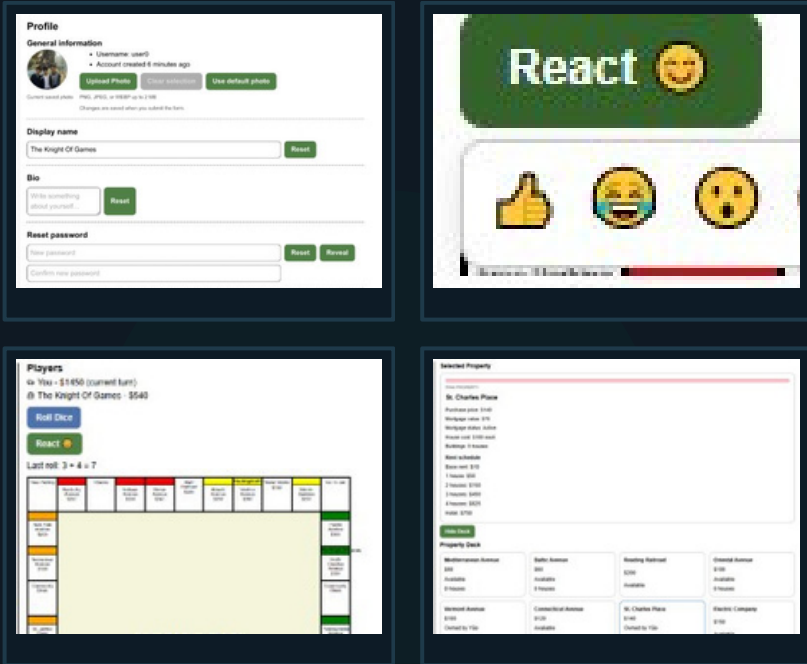


Three-tier architecture: React frontend, Express/Socket.io backend, Keyv key-value store backed by MongoDB in production. Shared package enforces type safety at compile time.

FUTURE WORK

- Invite privacy settings (friends only, anyone, nobody)
- Reaction preferences per user
- Profile picture privacy toggle
- Mid-game property trading between players
- Persistent leaderboard across sessions
- Additional Monopoly rule variants and presets

SCREENSHOTS



GameNite Upgrades: Matchmaking, Leaderboards, and Friends

Team 413: Motto Sereeyothin, Hanfu Yao, Qingnan Li, Miguel Chica

Project Description

Our team added ranked and casual automated matchmaking queues to GameNite for Nim and Number Guesser, using an ELO rating system to pair players of similar skill and track progression over time. Players can view their match history, ELO trends, and compete on global leaderboards to see how they stack up against others. A friend system allows players to compare stats, message each other, and view a friends-only leaderboard.

Tech Stack and Design Decisions

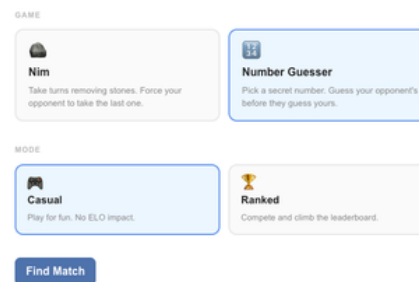
The project is built on a client/server/shared monorepo, using React on the frontend, Express on the backend, and MongoDB for persistence, with the shared package providing types and Zod schemas to keep both sides aligned. WebSockets handle real-time features like matchmaking events, disconnect handling, and live chat, while leaderboards, ranked history, and friend operations use REST endpoints for easier fetching and testing. The backend follows a controller/service/repository pattern, with feature logic in dedicated service files (e.g., `matchmaking.service.ts`, `elo.service.ts`), to keep route handlers thin and business logic centralized.

Design Decisions

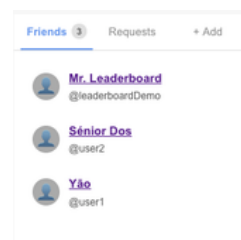
- **Consistent with existing patterns:** Extended the starter codebase and kept the repo/model structure, keeping controllers thin for readable route handling and placing feature logic in service files.
- **WebSockets:** for real-time features: Used for matchmaking events, disconnect handling, live chat, and game updates.
- **REST for static features:** Leaderboards, ranked history, and friend operations use REST endpoints, making them easier to fetch, test, and display.
- **In Memory Queue:** One notable tradeoff is that queue state is held in memory rather than a shared queue system — acceptable for a single-deployment class project, but a bottleneck if scaled further.

Find a Match

Choose a game and mode, then jump in.

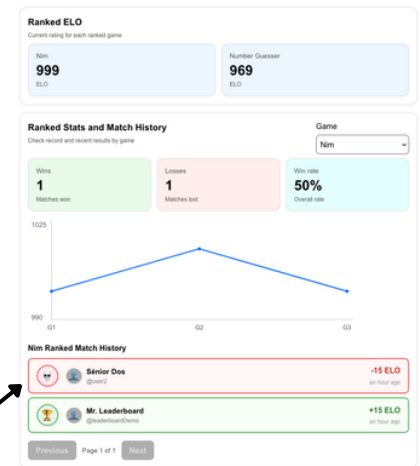


The 'Find a Match' Page. Choose your game and mode, then just wait until you find a match.



Friends list and actions

Your ranked stats



Leaderboards

Rank	Player	ELO	Record
#1	imgodatgames	1059	4-0
#2	Sénior Dos	1017	1-0
#3	totallyNotADemoUser2	1016	1-0
#4	totallyNotADemoUser4	1016	1-0

What's Next?

Next, we would expand both the ranked and social systems. Interesting additions would be seasonal resets, placement matches, private game invites between friends, and more advanced profile or messaging features. We would also want to refactor the matchmaking queue into a shared persistent system so the app could scale more reliably beyond a single class-project deployment.

Access Our Project

Repository link: <https://github.com/neu-cs4530/spring26-project-spring26-project-413>

Deployed link: <https://spring26-project-spring26-project-413.onrender.com/>



GameNite: Group 414

Automatic Matchmaking

signed in as The Knight Of Gam

Match found! Joining game in 2...

All games

Create New Game

Find a Match

Nim Find Match

waiting, 1 player [A name of Nim](#)

active, 4 players [A name of Number Guesser](#)

done [A name of Nim](#)

— Select a game — Find Match

Markdown Language support

Create new post

NEW FORUM POST

Header

BOLD TEXT

normal text here

Live markdown preview is on the right

[@img\(png/pngsum/photos/200\)](#)

Create

All games

Create New Game

Find a Match

— Select a game — Find Match

Host Private Games

[Private (Invite Only)] Nim

Game room created just now

1 / 2 players in queue

Close

Invite Code: e19728c4-6f10-433a-8515-a7bdba8a2dc8

Yao

The Knight Of Games

Invited Players:

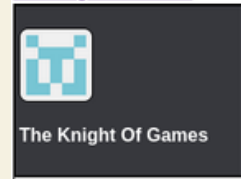
you

you are player #1

Waiting for more players...

Custom Pop-up Profiles

The Knight Of Games



The Knight Of Games

Chat with Friends

Yao

The Knight Of Games

Hi Friend!

you entered chat a minute ago

HELLO!!

Adding Friends

Friends List

Friends

Yao Remove Friend

Incoming Requests

Yao Accept Decline

Outgoing Requests

Yao Cancel Request

PROJECT DESCRIPTION:

GameNite is a web platform combining a community forum with multiplayer games, creating a space to play and discuss. This project extends the platform with three major improvements: an automatic matchmaking queue so players can find opponents without manually browsing lobbies, social features including friends lists, private messaging, and profile customization, and rich forum communication through full Markdown and image support.

TECHNOLOGY STACK

These enhancements are built with a TypeScript back-end, React front-end, Socket.io for real-time events, and MongoDB for persistent storage, deployed on Render.

WHAT'S NEXT

Future work includes messaging privacy controls and queue indicators for friends. Direct image uploads are partially implemented and ready to complete. Longer term, a notification system for friend activity and match invites could significantly improve platform engagement.

LINKS

Demo:

bit.ly/4cgSBX7

Repo:

<https://github.com/neu-cs4530/spring26-project-spring26-project-group-414>

